

Spring Boot

JWT(JSON Web Token)



Spring



Boot



Security

스프링 boot API

<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>

SPA 구조

MPA와 SPA를 간단히 비교해 보도록 하겠습니다.

MPA(Multi Page Application)

예시) JSP + Spring MVC 구조

브라우저 → 서버 요청 → 서버가 HTML 생성 → 브라우저 전체 새로 고침

페이지 이동마다 전체 HTML 다시 받음

화면이 계속 깜빡임

서버가 View까지 담당

SPA(Single Page Application)

예시) React

브라우저에 index.html 한 번 로딩

↓

이후에는 JS가 화면만 동적으로 변경

↓

서버와는 JSON 데이터만 주고 받음

페이지 전체 새로 고침 없음

속도 빠름

서버는 API만 제공

SPA 구조

‘단 하나의 HTML 페이지로 동작하는 웹 애플리케이션 구조’를 말합니다.

SPA 프레임워크

React
Vue.js
Angular

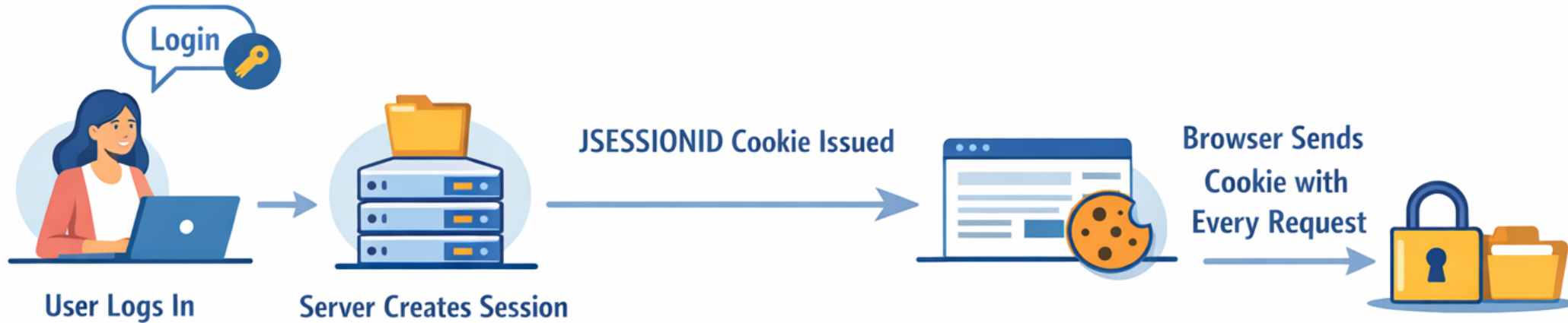
주요 특징

항목	설명
페이지	1개
화면 전환	JS가 담당
서버 역할	데이터(API) 제공
인증 방식	JWT 사용 많음
장점	빠름, 앱 같은 UX
단점	초기 로딩 느릴 수 있음

Session 인증 방식

Session 방식은 서버가 로그인한 사용자의 정보를 직접 저장하는 인증 방식입니다.

Session-Based Authentication



Server Stores User State

Stateful Authentication Model

Used in Traditional Web Apps

JWT

JWT 인증 방식

JWT 방식은 사용자가 로그인하면 서버에서 JSON Web Token을 발급해주는 인증 방식입니다.

JWT-Based Authentication



Server Doesn't Store User State

Stateless Authentication Model

Common in SPA:
e.g., React + Spring Boot



Session 방식과 JWT 방식

Session 방식과 JWT 방식을 비교해 봅니다.

Session 방식

전통적인 로그인 방식(Session 방식)

브라우저 → 로그인 요청

서버 → 세션 생성

서버 메모리에 사용자 저장

브라우저 ← 세션 ID 쿠키

이 방식은 서버가 사용자를 기억합니다.

단점:

서버 메모리 사용

서버가 여러 대일 경우 세션 공유 필요

JWT 방식

JWT 방식(Stateless 방식)

브라우저 → 로그인 요청

서버 → JWT 생성

브라우저 ← JWT 저장

이후 요청할 때:

브라우저 → JWT를 Header에 담아서 전송

서버 → JWT 검증 후 통과

✓ 서버는 사용자를 저장하지 않습니다

✓ 토큰만 검증합니다

Session 방식과 JWT 방식

Session 방식과 JWT 방식을 비교해 봅니다.

비교

구분	Session	JWT
서버 저장	O	X
확장성	낮음	높음
모바일 적합	보통	매우 적합
로그 아웃	쉬움	어려움

JWT는 서버가 상태를 관리하지 않기 때문에, 로그 아웃 시 서버에서 세션을 끊는 개념이 없고 클라이언트가 토큰을 삭제하는 방식으로 처리되기 때문에 로그 아웃이 어렵다고 표현합니다.

Stateless

서버가 사용자 상태 정보를 저장하면 Stateful이라고 하고, 그렇지 않으면 Stateless라고 합니다.

비교

구분	Stateful	Stateless
기본 개념	서버가 사용자 상태를 저장	서버가 사용자 상태를 저장하지 않음
로그인 상태 저장 위치	서버 메모리/세션 저장소	클라이언트(JWT 등)에 저장
대표 인증 방식	세션(Session), JSESSIONID	JSON Web Token
서버 메모리 사용	많음(사용자 수만큼 세션 유지)	거의 없음
요청 처리 방식	이전 상태를 참고하여 처리	매 요청이 독립적으로 처리됨
서버 확장성(Scale-out)	어려움(세션 공유 필요)	매우 좋음(아무 서버나 처리 가능)
클라우드 환경 적합성	보통(Redis 등 필요)	매우 적합
모바일 앱 연동	불편	매우 편리
보안 특징	세션 탈취 위험	토큰 탈취 위험
로그아웃 처리	서버에서 세션 삭제	클라이언트 토큰 삭제(또는 블랙리스트 관리)
구조 복잡도	단순(전통 웹에 적합)	구조 설계 필요(토큰 관리 전략 필요)
주 사용 환경	전통 JSP / MVC	SPA + REST API

JWT의 장점/단점

JWT의 장점과 단점에 대하여 살펴 보겠습니다.



Pros & Cons of JWT

Advantages

- ✓ Stateless (Server Does Not Store Sessions)
- ✓ Highly Scalable
- ✓ Suitable for Mobile Apps
- ✓ Good for Microservices

Disadvantages

- ✗ Risk of Token Theft
- ✗ Difficult to Log Out (Token Valid Until Expiry)

JWT의 특징

JWT의 특징에 대하여 살펴 보겠습니다.

JWT의 특징

지금 단계에서 꼭 이해해야 할 핵심

JWT는 단순 문자열이 아니다.

내부에 사용자 정보가 들어 있다.

서명 때문에 위변조 불가능.

서버는 토큰만 검증한다.

JWT의 구조

JWT는 Header | Payload | Signature의 3가지 영역으로 구성이 됩니다.

구성 요소

항목	설명
Header	토큰 타입, 서명 알고리즘 정보(어떤 알고리즘으로 서명했나?)
Payload	실제 데이터(사용자 정보, 권한 등), 여기서 Claims 가 들어갑니다
Signature	위변조 방지용 서명 으로 SECRET_KEY(문자열 변수)로 암호화해서 생성해 줍니다.

Token 예시

점을 기준으로 [Header] . [Payload] . [Signature]으로 구성이 되어 있습니다.

```
eyJhbGciOiJIUzI1NiJ9.eyJyY2xlljoiVVNFUiJ9.9TtnHdAWS6XX9iLYOHafKVJmw3_JAJqg3yHT98VrEpM
```

Header

```
eyJhbGciOiJIUzI1NiJ9
```

이 부분은 토큰의 메타 정보를 담고 있습니다.
Base64URL로 인코딩되어 있으며, 디코딩하면 다음과
같은 JSON 형태가 됩니다.



```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

JWT의 구조

JWT는 Header | Payload | Signature의 3가지 영역으로 구성이 됩니다.

Payload

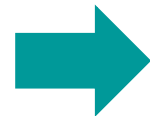
eyJyY2xlljoiVVNFUiJ9

이 부분은 '사용자 정보(클레임)'를 담고 있습니다.

디코딩하면 다음과 같습니다.

여기서 "sub", "iat", "exp" 같은 것들이 Claims입니다.

개발자가 createToken()에서 입력 해주는 값이 여기에 들어갑니다.



```
{
  "sub": "user@email.com", // 사용자
  ID(Registered)
  "name": "Jane Doe", // Public/Private
  "role": "admin", // Private
  "iat": 1670000000, // 발급 시간
  "exp": 1670003600 // 만료 시간
}
```

Signature(위변조 방지)

9TtnHdAWS6XX9iLY0HaFKVJmw3_JAJqg3yHT98VrEpM

이 부분은 토큰의 무결성을 검증하기 위한 값입니다.

다음과 같은 코드가 있기 때문에 토큰을 마음대로 수정할 수 없습니다.



```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secretKey
)
```

Jwts 클래스

JWT를 생성하고 파싱(해석)하기 위한 빌더/파서 생성 도구 클래스입니다.

클래스는 JJWT 라이브러리 안에 들어 있습니다.

JSON Web Token을 자바에서 쉽게 만들고 검증하도록 도와주는 유틸리티 진입점(Entry Point) 역할을 합니다.

Jwts.builder()를 호출하면 JWT 토큰을 만들기 위한 빌더 객체가 만들어집니다.

builder()

이 빌더를 통해서 토큰에 넣을 Claim 설정(setSubject(), claim(), setExpiration() 등), 서명 방식 및 비밀키 설정(signWith()), 최종 토큰 생성(compact())를 단계별로 지정할 수 있습니다.

JWT를 만들기 위한 빌더 객체 생성

내부적으로 하는 일

- Header 생성
- Payload 생성
- Base64 인코딩
- HS256 서명 생성
- 최종 문자열 조합

Jwts.builder()

```
.setSubject(email) // 토큰 주인공_subject 설정
.setIssuedAt(new Date()) // 토큰 발급 시간
.setExpiration(new Date(System.currentTimeMillis() + EXPIRATION)) // 만료 시간 설정
.signWith(getSigningKey(), SignatureAlgorithm.HS256) //서명 방식(HS256 알고리즘) 및 비밀
키 설정
.compact(); // 토큰 생성
```

Jwts 클래스

JWT를 생성하고 파싱(해석)하기 위한 빌더/파서 생성 도구 클래스입니다.

parserBuilder()

JWT를 해석하고 검증하는 파서 생성

내부적으로 하는 일

- 토큰 분리(HEADER / PAYLOAD / SIGNATURE)
- 서명 검증
- 만료 시간 검증
- Payload 반환

Claims 인터페이스

JWT 안에 담기는 사용자 정보(누가 주인, 어떤 권한)의 데이터 조각을 의미합니다.

JWT 안에 들어 있는 '사용자 정보나 토큰 정보'를 담은 데이터라고 이해하시면 됩니다.

Claims 종류

항목	설명	예시
Registered Claims(등록된 클레임)	JWT 표준에서 미리 정의된 키값들	
Public Claims(공용 클레임)	누구나 정의해서 사용할 수 있는 키값. 단, 충돌 방지를 위해 URI 같은 네임스페이스 권장.	role(권한), department
Private Claims(개인 클레임)	특정 시스템에서만 사용하는 사용자 정의 정보.	userId, isPremiumUser

Registered Claims

항목	설명
iss	토큰 발급자(Issuer)
sub	토큰 주제(Subject, 보통 사용자 ID)
aud	토큰 대상자(Audience)
exp	만료 시간(Expiration)
iat	발급 시간(Issued At)
nbf	유효 시작 시간(Not Before)

OncePerRequestFilter 클래스

하나의 HTTP 요청(request)당 딱 한 번만 실행되도록 보장해주는 추상 필터 클래스입니다.

모든 HTTP 요청에 대하여 컨트롤러가 실행되기 전에 가로 채기 하여 JWT를 검사하는 역할을 합니다.

Spring Security에서 JWT 인증, 로그, CORS 처리 등을 할 때 자주 사용됩니다.

중복 실행 방지를 위하여 한 요청(Request)에 대해 딱 한 번만 실행되는 필터입니다.

추상 메소드 `doFilterInternal()`를 반드시 오버라이딩해야 합니다.

의존성 추가

JWT와 관련된 의존성 정보를 추가합니다.

pom.xml

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

JwtTokenProvider 클래스

JWT 토큰을 생성하고, 검증하고, 토큰에서 사용자 정보를 추출하는 핵심 클래스입니다.

해당 클래스는 JWT를 생성하고, 검증하고, 토큰에서 사용자 이메일을 꺼내는 담당자 역할을 합니다.

구성 요소

항목	설명
SECRET_KEY	JWT 암호화/서명(Signature)을 만들 때 사용하는 비밀 키로 토큰 위조 방지 목적 즉, 클라이언트가 토큰을 조작하더라도 서버에서 SECRET_KEY로 검증을 다시 합니다.
EXPIRATION	토큰의 유효 만기 시간(밀리초 단위) 예시) 1000 * 60 * 60; // 1시간
createToken()	JWT 생성하기
getSigningKey()	SECRET_KEY를 암호화 Key 객체로 변환, HS256은 최소 256bit(32바이트) 키 필요
getEmail()	JWT 토큰에서 이메일 꺼내기
getClaims(token)	토큰 정보를 사용하여 Claims 정보를 추출
validateToken()	토큰 유효성 검사(토큰 정상 여부, 만료 여부, 위조 여부 등)

JwtAuthenticationFilter 클래스

JWT 인증을 실제로 수행해주는 필터(Filter) 입니다.

매번 요청마다 JWT 토큰을 검사해서, 정상이면 로그인된 사용자로 만들어 주는 역할을 합니다.

Spring Security의 인증 흐름 안에서 동작하며, JwtTokenProvider와 함께 JSON Web Token 기반 인증을 완성합니다.

구성 요소

항목	설명
jwtTokenProvider	토큰 검증과 이메일을 추출하는 객체(외부에서 주입)
doFilterInternal()	모든 요청이 들어올 때 컨트롤러 보다 먼저 실행이 되는 핵심 로직

doFilterInternal 메소드가 하는 일

요청 헤더에서 Authorization 값을 가져 옵니다.
값이 존재하고 "Bearer "로 시작하는지 확인합니다.
"Bearer "를 제거하여 JWT 토큰만 추출합니다.
토큰의 유효성을 검증합니다. (validateToken)
토큰에서 사용자 이메일을 추출합니다. (getEmail)
토큰의 claims에서 role 값을 추출합니다.
role을 기반으로 권한 객체(GrantedAuthority)를 생성합니다.
이메일 + 권한 정보를 이용해 Authentication 객체를 생성합니다.
생성한 Authentication 객체를 SecurityContext에 저장합니다.
다음 필터 또는 컨트롤러로 요청을 전달한다 (filterChain.doFilter)